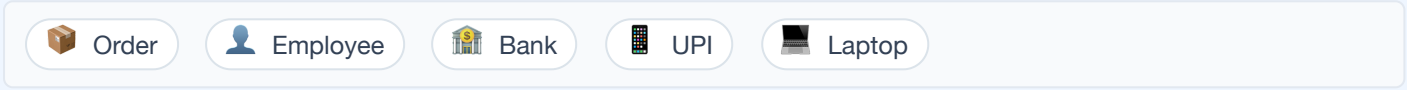
Data Structures Overview — Lists, Tuples, Sets, Dicts

Module: Core Python · Collections capstone
Previous module: Dictionaries
Next module: Functions Decorators Generators

Module Outcome:

- Pick list vs tuple vs set vs dict the way a shop picks a bag vs a filing cabinet
- Say *why* Swiggy cart is a list and SKU lookup is a dict — in one sentence
- Spot the wrong container before it makes the next module painful

Python gives you four built-in **containers** for grouping data. Choosing the wrong one makes code slower, bug-prier, or harder to read — like using a shopping bag when you need a labelled filing cabinet.



Where used?

- **Swiggy** — cart (list), SKU lookup (dict), unique tags (set), GPS drop (tuple)
- **HDFC Bank** / **PhonePe** — transaction history list, account dict by ID
- **Employee Management Portal** — employee-by-id dict, unique active-ID set
- College apps — undo stack for form edits, ticket queues
- **Apollo Hospitals** — support ticket FIFO queue, banned-user set

Quick decision guide:

```
Need key → value lookup?     → dict
Need unique items only?      → set
Need fixed record (no edits)? → tuple
Need ordered, changeable list? → list
```

Same employee ID problem — four views:

Structure	Example	Best when
List	`[101, 102, 105]`	Order matters; duplicates allowed; index access
Tuple	`(101, "Ananya", "Eng")`	Fixed row; return multiple values
Set	`{101, 102, 105}`	Unique IDs only; fast "is member?"
Dict	`{101: "Ananya", 102: "Dev"}`	Lookup name by ID instantly

You will use **all four** in full-stack work: scripts, JSON APIs, cart logic, and interview screens.

Concepts and Definitions

Term	Definition
Data structure	Way to organize data for storage and access.
Sequence	Ordered items with indexing — list, tuple, str.
Mapping	Keys map to values — dict.
Mutable	Can change after creation — list, dict, set.
Immutable	Cannot change — tuple, str, frozenset.
Stack (LIFO)	Last In, First Out — like undo history in editor.
Queue (FIFO)	First In, First Out — like ticket counter line.
Big-O	How runtime grows as data size **n** grows — interview intuition.
O(1)	Constant time — size barely affects speed.
O(n)	Linear — double data ≈ double time.
O(n²)	Quadratic — nested loops over same data.

Comparison Table — List vs Tuple vs Set vs Dict

Feature	List	Tuple	Set	Dict
Syntax	`[ ]`	`( )`	`{ }` / `set()`	`{k: v}`
Ordered	Yes	Yes	No*	Yes (3.7+)
Mutable	Yes	No	Yes	Yes
Duplicates	Yes	Yes	No	Keys unique
Index `[i]`	Yes	Yes	No	By key only
Use case	Cart items, marks	Coordinates, DB row	Unique tags, dedupe	Records, JSON, lookup

*Sets are **unordered** — never depend on set iteration order (dicts keep insertion order since 3.7; that rule does not apply to sets).

Step-by-Step Notes with Annotated Python 3 Code

Step 1 — Scenario chooser (walkthrough)

```
# SCENARIO 1: E-commerce cart — order matters, items added/removed
cart = ["Notebook", "Pen", "USB Cable"]          # LIST

# SCENARIO 2: GPS delivery point — fixed lat/lon
drop_point = (12.9716, 77.5946)                  # TUPLE

# SCENARIO 3: Unique product categories on homepage filter
```

```
categories = {"Electronics", "Books", "Electronics"}  # SET → one "Electronics"

# SCENARIO 4: Employee lookup by ID
employees = {
    101: {"name": "Ananya", "dept": "Engineering"},
    102: {"name": "Dev", "dept": "Support"},
}                                                  # DICT

print(employees[101]["name"])                      # Ananya — O(1) average lookup
```

Print each scenario paired with the best data structure (list, tuple, set, or dict).

```
scenarios = {
    "cart items": "list",
    "GPS point": "tuple",
    "unique tags": "set",
    "employee-by-id": "dict",
}
for scenario, structure in scenarios.items():
    print(f"{scenario} → {structure}")
```

Expected Output:

```
cart items → list
GPS point → tuple
unique tags → set
employee-by-id → dict
```

Interview: Start answers with the decision guide — key→value? dict · unique? set · fixed row? tuple · ordered mutable? list.

Create list, tuple, set, and dict examples for a Swiggy canteen and print all four.

```
cart = ["Chai", "Samosa"]                 # list
coord = (17.3850, 78.4867)                # tuple (lat, lon)
tags = {"veg", "spicy"}                   # set
menu = {"Chai": 20, "Samosa": 25}          # dict
print(cart, coord, tags, menu)
```

Step 2 — When NOT to use each structure

```
# BAD: list for membership at scale
banned_users = [1001, 1002, 1003, /* ... 1 lakh IDs ... */]
# if user_id in banned_users:   # O(n) — slow for huge list

# BETTER: set for membership
banned_users = {1001, 1002, 1003}
# if user_id in banned_users:   # O(1) average

# BAD: dict when you only need ordered names (no lookup key)
# names = {"0": "A", "1": "B"}  # awkward

# BETTER: list
names = ["Aisha", "Dev", "Meera"]

# BAD: list for fixed config that should not change
```

```
# API_VERSION = ["v2"]          # someone might .append by mistake

# BETTER: tuple
API_VERSION = ("v2",)
```

Note:
- Wrong: Using a list for many `in` checks (O(n) each time); a dict when an order-only list suffices; a tuple for data you must update.
- Prefer: Set for membership; a simple list when you only need order; list or dict when data must change.

Store banned user IDs in a set and check membership for ID 1002.

```
banned_users = {1001, 1002, 1003}
print(1002 in banned_users)
```

Expected Output:

```
True
```

Create a one-item API version tuple and print it (tuples cannot be appended).

```
API_VERSION = ("v2",)
# API_VERSION.append("v3")  # AttributeError — tuples have no append
print(API_VERSION)
```

Expected Output:

```
('v2',)
```

Step 3 — Stack using list (LIFO)

```
# Browser back button / undo stack
history = []

def visit(page):
    history.append(page)              # push
    print(f"Visited: {page}")

def go_back():
    if history:
        left = history.pop()          # pop from end — LIFO
        print(f"Back from: {left}")
        print(f"Now at: {history[-1] if history else 'Home'}")
    else:
        print("Nothing to go back to")

visit("home")
visit("products")
visit("cart")
go_back()                             # Back from: cart → Now at: products
go_back()                             # Back from: products → Now at: home
```

Note:
- Wrong: Stack pop from the front — wrong end; not LIFO.
- Prefer: `pop()` from the end.

Push three pages onto a history stack, pop twice, and print the remaining history.

```
history = []
for page in ("home", "products", "cart"):
    history.append(page)
history.pop()
history.pop()
print(history)  # ['home']
```

Expected Output:

```
['home']
```

Remember: Stack = **LIFO** — push/pop from the **same end** (`append` / `pop()`). Queue = **FIFO** — enqueue at end, dequeue from front (`pop(0)` or `deque.popleft()`).

Simulate form undo with append and pop, then print the reverted and removed values.

```
undo = []
undo.append("Priya")
undo.append("Priya Nair")
current = undo.pop()
previous = undo.pop() if undo else ""
print("undid to:", previous)
print("was:", current)
```

Expected Output:

```
undid to: Priya
was: Priya Nair
```

Step 4 — Queue using list (FIFO)

```
# Support ticket queue — first ticket handled first
ticket_queue = []

def enqueue(ticket_id):
    ticket_queue.append(ticket_id)    # add at rear

def dequeue():
    if ticket_queue:
        served = ticket_queue.pop(0)    # remove from front — FIFO
        print(f"Serving ticket: {served}")
    else:
        print("Queue empty")

enqueue("T-101")
enqueue("T-102")
enqueue("T-103")
```

```
dequeue()                           # T-101
dequeue()                           # T-102
```

Note: `pop(0)` on a list is **O(n)** because remaining items shift. For production queues, `collections.deque` is preferred — know the idea; you do not need to master deque yet.

```
from collections import deque

fast_queue = deque()
fast_queue.append("T-201")
fast_queue.popleft()                # O(1) — good for interviews to name
```

Note:
- Wrong: Queue pop from the end — that is LIFO, not FIFO.
- Prefer: `pop(0)` (or `deque.popleft()`).

Enqueue three tickets with a list, serve two from the front, and print the next ticket.

```
queue = []
for t in ("T-1", "T-2", "T-3"):
    queue.append(t)
queue.pop(0)
queue.pop(0)
print("Next:", queue[0] if queue else "empty")
```

Expected Output:

```
Next: T-3
```

Common Error: Using `pop()` (no index) for a queue — that removes the **last** item (LIFO), not the first customer in line.

Run the same ticket queue flow using `deque` and `popleft`, then print the next ticket.

```
from collections import deque

queue = deque()
for t in ("T-1", "T-2", "T-3"):
    queue.append(t)
queue.popleft()
queue.popleft()
print("Next:", queue[0] if queue else "empty")
```

Step 5 — Big-O intuition (lite — no proofs)

```
# O(1) — constant: dict key lookup (average case)
product = {"P001": 799, "P002": 1499}
price = product["P001"]

# O(n) — linear: scan list for one ID
ids = [101, 102, 105, 108]
target = 105
```

```
found = False
for i in ids:
    if i == target:
        found = True
        break

# O(n) — linear: membership in list
# "Pen" in cart_items

# O(1) average — membership in set
tags = {"sale", "new", "electronics"}
# "sale" in tags

# O(n²) — quadratic: nested loop over same list
marks = [78, 92, 85, 92]
pairs = 0
for i in range(len(marks)):
    for j in range(len(marks)):
        if marks[i] == marks[j]:
            pairs += 1
# Doubling list size ≈ 4× work — avoid in hot paths
```

Note:
- Wrong: Ignoring nested-loop cost (hidden O(n²)); memorizing Big-O with no job context.
- Prefer: Know your loop depth; tie complexity to cart, lookup, and dedupe examples in interviews.

Check membership of 105 in both a list and a set built from the same range.

```
ids_list = list(range(100, 120))
ids_set = set(ids_list)
print(105 in ids_list)
print(105 in ids_set)
```

Count equal mark pairs with nested loops and print the count.

```
marks = [78, 92, 85, 92]
count = 0
# O(n²) nested scan
for i in range(len(marks)):
    for j in range(i + 1, len(marks)):
        if marks[i] == marks[j]:
            count += 1
print(count)
```

Step 6 — Complexity cheat sheet (interview lite)

Operation	List	Tuple	Set	Dict
Index access `x[i]`	O(1)	O(1)	—	—
Key lookup	—	—	—	O(1)* avg
`item in coll`	O(n)	O(n)	O(1)* avg	O(1)* keys
Append / add	O(1)*	—	O(1)*	O(1)*
Insert middle	O(n)	—	—	—
pop(0) queue	O(n)	—	—	—

*Average case; simplified for entry-level interviews.

Print a loaded message after commenting Big-O costs for list `in`, set `in`, dict lookup, and `pop(0)`.

```
# list `in` → O(n)
# set `in` → O(1) average
# dict key lookup → O(1) average
# list pop(0) → O(n)
print("cheat sheet loaded")
```

Use a set of known SKUs and check membership for P002.

```
# Use a set (or dict keys) for 10,000 SKUs — average O(1) membership.
known_skus = {"P001", "P002", "P003"}  # imagine 10,000 entries
print("P002" in known_skus)
```

Step 7 — Combining structures (realistic app shape)

```
# Campus e-commerce: list of order dicts, each with list of item dicts
orders = [
    {
        "order_id": "ORD-1",
        "customer_id": 501,
        "items": [
            {"sku": "P1", "qty": 2, "price": 120},
            {"sku": "P2", "qty": 1, "price": 499},
        ],
        "status": "shipped",
    },
    {
        "order_id": "ORD-2",
        "customer_id": 502,
        "items": [{"sku": "P3", "qty": 5, "price": 15}],
        "status": "pending",
    },
]

# Unique SKUs sold (set comprehension)
skus_sold = {item["sku"] for order in orders for item in order["items"]}
print(skus_sold)                      # {'P1', 'P2', 'P3'}

# Total revenue — nested loops O(orders × items)
revenue = sum(
    item["qty"] * item["price"]
    for order in orders
    for item in order["items"]
)
print(f"Revenue: ₹{revenue}")
```

Collect unique SKUs from nested order items with a set comprehension and print them.

```
orders = [
    {"items": [{"sku": "P1", "qty": 2, "price": 120}, {"sku": "P2", "qty": 1, "price": 499}]},
    {"items": [{"sku": "P1", "qty": 1, "price": 120}, {"sku": "P3", "qty": 2, "price": 50}]},
]
skus = {item["sku"] for order in orders for item in order["items"]}
print(skus)
```

Sum qty × price across all order items and print total revenue.

```
orders = [
    {"items": [{"sku": "P1", "qty": 2, "price": 120}, {"sku": "P2", "qty": 1, "price": 499}]},
    {"items": [{"sku": "P1", "qty": 1, "price": 120}, {"sku": "P3", "qty": 2, "price": 50}]},
]
revenue = sum(item["qty"] * item["price"] for order in orders for item in order["items"])
print(f"₹{revenue}")
```

Expected Output:

```
₹959
```

Step 8 — Expense report: pick the right tool

```
# Raw expenses — list of dicts (JSON-like from API)
expenses = [
    {"id": 1, "category": "Food", "amount": 450},
    {"id": 2, "category": "Travel", "amount": 1200},
    {"id": 3, "category": "Food", "amount": 320},
]

# Index by id for fast update — dict
by_id = {e["id"]: e for e in expenses}
by_id[2]["amount"] = 1150             # corrected Travel expense

# Unique categories for filter dropdown — set
categories = {e["category"] for e in expenses.values()}
print(categories)                     # {'Food', 'Travel'}

# Category totals — dict aggregation
totals = {}
for e in expenses:
    cat = e["category"]
    totals[cat] = totals.get(cat, 0) + e["amount"]
print(totals)                         # {'Food': 770, 'Travel': 1150}
```

Index expenses by id, update one amount, and print that expense record.

```
expenses = [
    {"id": 1, "category": "Food", "amount": 450},
    {"id": 2, "category": "Travel", "amount": 1200},
]
by_id = {e["id"]: e for e in expenses}
by_id[1]["amount"] = 500
print(by_id[1])
```

Aggregate category totals from expenses and print totals plus the set of categories.

```
expenses = [
    {"id": 1, "category": "Food", "amount": 450},
    {"id": 2, "category": "Travel", "amount": 1200},
    {"id": 3, "category": "Food", "amount": 320},
]
totals = {}
for e in expenses:
```

```
    totals[e["category"]] = totals.get(e["category"], 0) + e["amount"]
print(totals)
print(set(totals))
```

Expected Output:

```
{'Food': 770, 'Travel': 1200}
{'Food', 'Travel'}
```

(Set print order may vary — membership/`set(totals)` keys are what matter.)

Practice Exercises

Basic

1. For each scenario, name the best structure: (a) shopping cart (b) unique email list (c) lat/lon (d) phone → contact.

Model answer:
- (a) Shopping cart (ordered items, duplicates OK, change qty) → **list** (or list of dicts).
- (b) Unique email list → **set**.
- (c) lat/lon fixed pair → **tuple** `(lat, lon)`.
- (d) phone → contact → **dict** keyed by phone.

2. Push `"home"`, `"about"`, `"contact"` on a stack; pop twice — what remains?

Model answer: Stack is LIFO. After pushes: bottom `home` → `about` → top `contact`. Pop → `contact`; pop → `about`. Remains: `["home"]`.

`python stack = [] stack.append("home") stack.append("about") stack.append("contact") stack.pop() stack.pop() print(stack) # ['home']`

3. Enqueue three ticket IDs; dequeue one — which is served first?

Model answer: Queue is FIFO. Enqueue `T1`, `T2`, `T3`; dequeue serves **`T1`** first.

`python queue = [] queue.append("T1") queue.append("T2") queue.append("T3") served = queue.pop(0) print(served) # T1 print(queue) # ['T2', 'T3']`

Intermediate

4. Given 1000 employee IDs, design structure to check if ID is active on every login — list or set?

Model answer: Prefer a **set** of active IDs. Membership `id in active_ids` is average O(1) for a set vs O(n) scan for a list. With 1000 IDs and frequent logins, set wins. Keep a dict only if you also need employee details keyed by ID.

5. Refactor list-of-lists student marks to list-of-dicts with keys `name`, `marks`.

Model answer:

```
rows = [["Ananya", 88], ["Dev", 72], ["Meera", 95]]
students = [{"name": name, "marks": marks} for name, marks in rows]
print(students)
```

6. Write `is_balanced_parens(s)` using stack for `"((()))"` and `"(()"` (strings module tie-in).

Model answer:

```
def is_balanced_parens(s):
    stack = []
    for ch in s:
        if ch == "(":
            stack.append(ch)
        elif ch == ")":
            if not stack:
                return False
            stack.pop()
    return len(stack) == 0

print(is_balanced_parens("((()))"))  # True
print(is_balanced_parens("(()"))     # False
```

Interview-style

7. Explain time complexity of `x in my_list` vs `x in my_set` in plain English.

Model answer: `x in my_list` may scan every element — work grows with list size (**O(n)**). `x in my_set` uses hashing — typically near-constant time (**O(1)** average), so large unique membership checks belong in a set.

8. Why is `pop(0)` on a large list slow? What alternative did we mention?

Model answer: `pop(0)` removes the first element and shifts every later index left — **O(n)**. For frequent queue-style dequeue, use `collections.deque` and `popleft()` (**O(1)**), or redesign so you only `pop()` from the end.

9. You have 10,000 products keyed by SKU — list of tuples or dict? Justify.

Model answer: Use a **dict** `{sku: product_info}`. Lookup by SKU is O(1) average vs scanning a list of tuples O(n). With 10,000 products and many cart/search lookups, dict is the clear choice; list of tuples only if you mostly iterate in order and rarely look up by SKU.

Mini-Project (Assignment): Collection Toolkit Demo

Title: Stack, queue, and structure chooser in one file
Lab kit (build here): 15_Lab_Assets/12_structure_chooser/

What to build

One Python file with functions and a menu:

```
def demo_stack():
    """Undo history for form fields."""
    ...

def demo_queue():
    """Print job queue."""
    ...

def choose_structure(scenario: str) -> str:
    """Return 'list'|'tuple'|'set'|'dict' with one-line reason."""
    scenarios = {
        "cart": "list",
        "coordinates": "tuple",
        "unique_tags": "set",
        "user_profile_by_id": "dict",
    }
```

```
    return scenarios.get(scenario, "dict — default for lookup records")

if __name__ == "__main__":
    print("Stack demo:"); demo_stack()
    print("Queue demo:"); demo_queue()
    for s in ["cart", "coordinates", "unique_tags", "user_profile_by_id"]:
        print(f"{s} → {choose_structure(s)}")
```

Learners extend `choose_structure` with 5 custom scenarios from their domain (HR, ecommerce, expenses).

Stretch

- Add a `deque`-based queue demo and note O(1) `popleft` in a comment.
- Add scenarios: undo stack for ATM PIN retries, unique visitor emails.

Done when

- [ ] Lab kit structure-chooser + undo stack ticket passes
- [ ] `demo_stack` and `demo_queue` run from `__main__`
- [ ] `choose_structure` covers at least 4 built-ins + 5 custom scenarios

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. JSON from an API — which Python type on arrival? (dict / list via `json.loads`.)
2. When would you convert list → set → list deliberately?
3. Which structure fits “lookup student name by roll number” best, and why?
4. Is O(1) always faster than O(n) for tiny n? (Constant overhead — intuition only.)
5. Real product: where would you use a queue in an e-commerce site? (Order processing, email jobs.)
6. **Difference list vs tuple?**
   A: Mutability; tuple hashable if elements hashable.
7. **When use set?**
   A: Uniqueness, membership, set math (common skills, tags). Sets are unordered — never depend on iteration order.
8. **Dict vs list of dicts?**
   A: Dict for keyed lookup; list when you have an ordered collection of records without a single key index.
9. **Implement queue in Python?**
   A: `append` + `pop(0)` or `collections.deque`.
10. **What is O(n)?**
    A: Work grows roughly proportionally with input size; one full pass through data.

Next module: `03_Advanced_Python/01_Functions_Decorators_Generators.md` — reusable logic for cleaner collection code.